

Name: Sneha Gopal Bhalekar

Roll_no: cs05

Aim

To implement Support Vector Machine (SVM) for classification of the Iris dataset and visualize its decision boundary and compare it with a normal linear classification boundary.

Double-click (or enter) to edit

Objectives

1. To understand the concept of Support Vector Machine.
2. To implement SVM using Python.
3. To visualize the SVM decision boundary.
4. To identify the support vectors.
5. To compare the SVM boundary with a normal linear classification boundary.
6. To evaluate the performance of the classifiers.

Relevant Theory

Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification.

SVM finds a decision boundary that separates different classes while trying to maximize the distance, called the **margin**, between the classes.

The points closest to the decision boundary are called **support vectors**.

For a linear SVM, the decision boundary can be represented as:

$$w^T x + b = 0$$

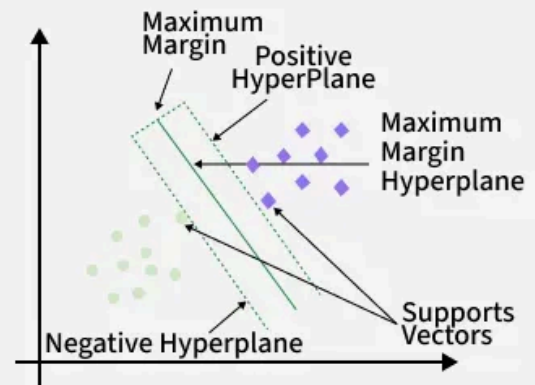
where:

- w = weight vector
- x = input features
- b = bias



Support Vectors & Hyperplane

- Support Vectors are the closest data points to the hyperplane that define the class boundary.
- A hyperplane is a plane that separates different classes.



Kernel

SVM can use different kernels to create decision boundaries.

Common kernels are:

- Linear
- Polynomial
- RBF
- Sigmoid

In this experiment, we mainly use the **Linear SVM** and **RBF SVM**.

Dataset Information

Get the IRIS dataset from Kaggle using below link

<https://www.kaggle.com/datasets/uciml/iris>

The Iris dataset contains measurements of Iris flowers belonging to three classes:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

The four features are:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

For visualization, we will use:

Petal Length and Petal Width.

✓ Task 1: Import Libraries and Load Dataset

Load the Iris dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv('/content/Iris.csv')

# Display first 5 rows
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	
0	1	5.1	3.5	1.4	0.2	Iris-setosa	
1	2	4.9	3.0	1.4	0.2	Iris-setosa	
2	3	4.7	3.2	1.3	0.2	Iris-setosa	
3	4	4.6	3.1	1.5	0.2	Iris-setosa	
4	5	5.0	3.6	1.4	0.2	Iris-setosa	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

✓ Task 2: Explore the Dataset

```
# Display dataset information
print("Dataset Shape:", df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns)

# Display dataset information
```

```

print("\nDataset Information:")
df.info()

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Display statistical summary
print("\nStatistical Summary:")
print(df.describe())

# Display class distribution
print("\nClass Distribution:")
print(df['Species'].value_counts())

```

Dataset Shape: (150, 6)

Column Names:

```

Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')

```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 150 entries, 0 to 149

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Id	150 non-null	int64
1	SepalLengthCm	150 non-null	float64
2	SepalWidthCm	150 non-null	float64
3	PetalLengthCm	150 non-null	float64
4	PetalWidthCm	150 non-null	float64
5	Species	150 non-null	object

dtypes: float64(4), int64(1), object(1)

memory usage: 7.2+ KB

Missing Values:

```

Id          0
SepalLengthCm  0
SepalWidthCm  0
PetalLengthCm  0
PetalWidthCm  0
Species      0
dtype: int64

```

Statistical Summary:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```

Class Distribution:
Species
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64

```

✓ Task 3: Select Features and Target

Remove the `Id` column.

Select Remaining Features

```

# Remove Id column
df = df.drop('Id', axis=1)

# Select input features
X = df[['SepalLengthCm', 'SepalWidthCm',
        'PetalLengthCm', 'PetalWidthCm']]

# Select target
y = df['Species']

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())

```

```

Features:
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0              5.1            3.5           1.4            0.2
1              4.9            3.0           1.4            0.2
2              4.7            3.2           1.3            0.2
3              4.6            3.1           1.5            0.2
4              5.0            3.6           1.4            0.2

```

```

Target:
0  Iris-setosa
1  Iris-setosa
2  Iris-setosa
3  Iris-setosa
4  Iris-setosa
Name: Species, dtype: object

```

✓ Task 4: Split and Scale the Data

Divide the dataset into training and testing sets.

```
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

# Standardize the features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nData scaling completed.")
```

```
Training data: (120, 4)
Testing data: (30, 4)
```

```
Data scaling completed.
```

✓ Task 5: Implement Linear SVM

Create an SVM model using a **linear kernel**.

Make predictions.

```
# Create Linear SVM model
linear_svm = SVC(kernel='linear', random_state=42)

# Train the model
linear_svm.fit(X_train_scaled, y_train)

# Make predictions
y_pred_svm = linear_svm.predict(X_test_scaled)

print("Linear SVM predictions:")
print(y_pred_svm)
```

```
Linear SVM predictions:
['Iris-setosa' 'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor'
 'Iris-setosa' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
 'Iris-virginica' 'Iris-versicolor' 'Iris-virginica' 'Iris-virginica'
 'Iris-virginica' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
 'Iris-setosa' 'Iris-virginica' 'Iris-versicolor' 'Iris-virginica'
 'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor' 'Iris-setosa'
 'Iris-virginica' 'Iris-setosa']
```

✓ Task 6: Evaluate SVM

Calculate Accuracy

Calculate the accuracy of the SVM model.

Confusion Matrix

Generate the confusion matrix to evaluate the classification results.

Classification Report

Generate the classification report containing:

- Precision
- Recall
- F1-Score
- Support

```
accuracy = accuracy_score(y_test, y_pred_svm)

print("Linear SVM Accuracy:", accuracy)
print("Linear SVM Accuracy: {:.2f}%".format(accuracy * 100))
```

```
Linear SVM Accuracy: 1.0
Linear SVM Accuracy: 100.00%
```

```
cm = confusion_matrix(y_test, y_pred_svm)

print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]
```

```
print("Classification Report:")
print(classification_report(y_test, y_pred_svm))
```

```
Classification Report:
              precision    recall  f1-score   support

   Iris-setosa              1.00        1.00        1.00         10
  Iris-versicolor              1.00        1.00        1.00         10
   Iris-virginica              1.00        1.00        1.00         10

     accuracy                   1.00         30
    macro avg              1.00        1.00        1.00         30
```

weighted avg	1.00	1.00	1.00	30
--------------	------	------	------	----

Task 7: Visualize SVM Decision Boundary

For visualization, we use only two features:

- Petal Length
- Petal Width

Create the 2-D Dataset

Select **Petal Length** and **Petal Width** as the input features.

Split the Data

Divide the 2-D dataset into training and testing sets.

Scale the Data

Standardize the selected features before training the SVM model.

Train SVM

Train an SVM classifier using a **linear kernel**.

Plot Decision Boundary

Visualize the SVM decision boundary along with the data points.

```
# Select two features
X_2d = df[['PetalLengthCm', 'PetalWidthCm']]
y_2d = df['Species']

# Split the 2-D dataset
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X_2d, y_2d,
    test_size=0.2,
    random_state=42,
    stratify=y_2d
)

# Scale the data
scaler_2d = StandardScaler()

X2_train_scaled = scaler_2d.fit_transform(X2_train)
X2_test_scaled = scaler_2d.transform(X2_test)

# Train Linear SVM
svm_2d = SVC(kernel='linear', random_state=42)
```

```

svm_2d.fit(X2_train_scaled, y2_train)

# Create mesh grid
x_min = X2_train_scaled[:, 0].min() - 1
x_max = X2_train_scaled[:, 0].max() + 1

y_min = X2_train_scaled[:, 1].min() - 1
y_max = X2_train_scaled[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

# Predict for every point in the grid
Z = svm_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Convert class names into numbers for plotting
class_names = sorted(y2d.unique())
class_to_num = {name: i for i, name in enumerate(class_names)}

Z_num = np.vectorize(class_to_num.get)(Z)
y_train_num = np.array([class_to_num[x] for x in y2_train])

# Plot decision boundary
plt.figure(figsize=(9, 6))

plt.contourf(xx, yy, Z_num, alpha=0.3)

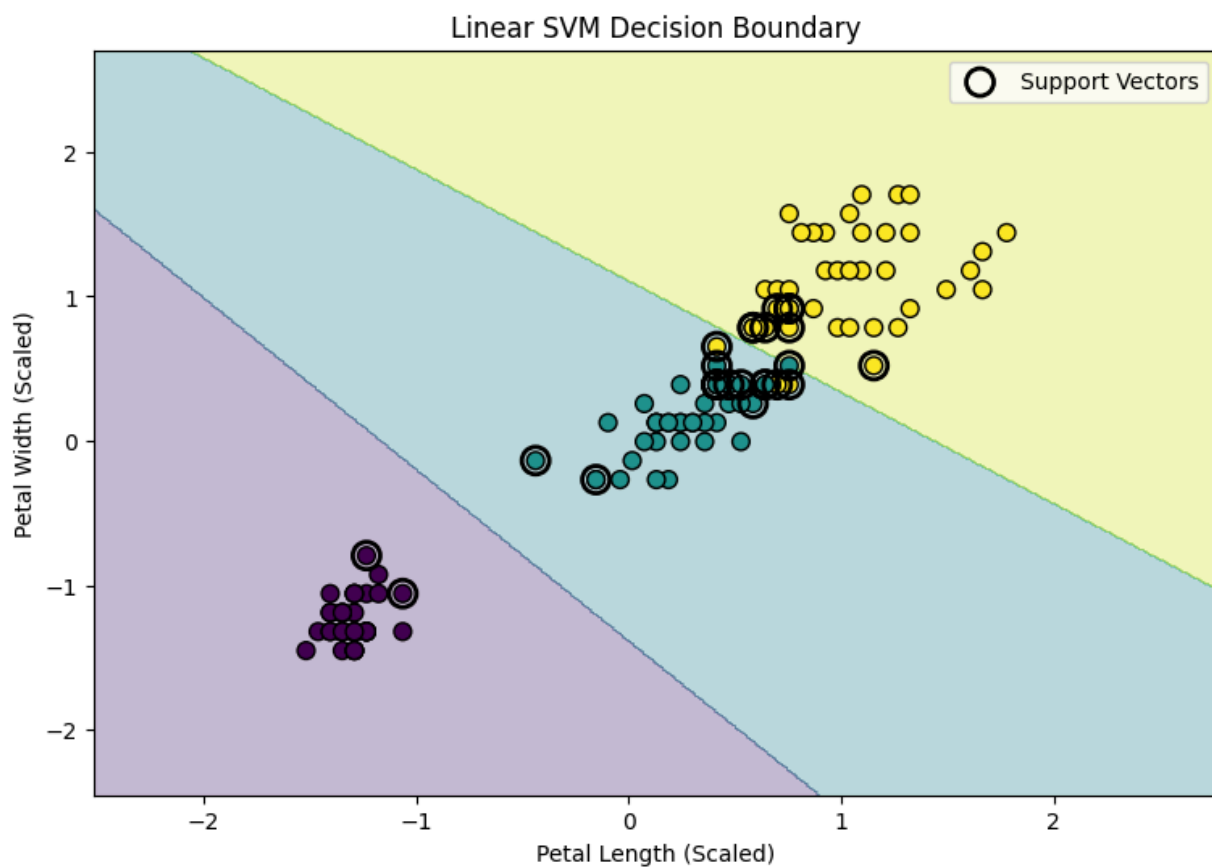
plt.scatter(
    X2_train_scaled[:, 0],
    X2_train_scaled[:, 1],
    c=y_train_num,
    edgecolor='black',
    s=60
)

# Highlight support vectors
plt.scatter(
    svm_2d.support_vectors_[:, 0],
    svm_2d.support_vectors_[:, 1],
    s=150,
    facecolors='none',
    edgecolors='black',
    linewidths=2,
    label='Support Vectors'
)

plt.xlabel('Petal Length (Scaled)')
plt.ylabel('Petal Width (Scaled)')
plt.title('Linear SVM Decision Boundary')
plt.legend()

```

```
plt.show()
```



```
print("Number of Support Vectors:",
      len(svm_2d.support_vectors_))
```

```
print("\nSupport Vectors:")
print(svm_2d.support_vectors_)
```

Number of Support Vectors: 25

Support Vectors:

```
[[-1.23340162 -0.79429223]
 [-1.06295073 -1.05723035]
 [ 0.58474127  0.25746024]
 [ 0.41429037  0.3889293 ]
 [ 0.64155823  0.3889293 ]
 [-0.15387928 -0.268416 ]
 [ 0.58474127  0.78333648]
 [ 0.75519216  0.52039836]
 [ 0.41429037  0.3889293 ]
 [ 0.47110734  0.3889293 ]
 [ 0.41429037  0.52039836]
 [ 0.41429037  0.3889293 ]
 [-0.43796411 -0.13694694]
 [ 0.5279243   0.3889293 ]
 [ 0.64155823  0.3889293 ]
 [ 0.41429037  0.65186742]
```

```
[ 0.6983752  0.91480554]
[ 0.75519216  0.3889293 ]
[ 0.75519216  0.91480554]
[ 0.75519216  0.78333648]
[ 0.75519216  0.91480554]
[ 0.6983752  0.3889293 ]
[ 1.15291092  0.52039836]
[ 0.64155823  0.78333648]
[ 0.58474127  0.78333648]]
```

✓ Task 8: Normal Linear Classification Boundary

For comparison, we use **Logistic Regression** as a normal linear classifier.

Train a Logistic Regression model using the same two features and visualize its decision boundary.

```
# Create Logistic Regression model
logistic_model = LogisticRegression(
    random_state=42,
    max_iter=200
)

# Train model
logistic_model.fit(X2_train_scaled, y2_train)

# Make predictions
y_pred_logistic = logistic_model.predict(X2_test_scaled)

print("Logistic Regression predictions:")
print(y_pred_logistic)
```

```
Logistic Regression predictions:
['Iris-setosa' 'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor'
 'Iris-setosa' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
 'Iris-virginica' 'Iris-versicolor' 'Iris-virginica' 'Iris-virginica'
 'Iris-virginica' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
 'Iris-setosa' 'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor'
 'Iris-virginica' 'Iris-virginica' 'Iris-versicolor' 'Iris-setosa'
 'Iris-virginica' 'Iris-setosa']
```

```
# Predict grid points
Z_logistic = logistic_model.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

Z_logistic = Z_logistic.reshape(xx.shape)

Z_logistic_num = np.vectorize(class_to_num.get)(Z_logistic)
```

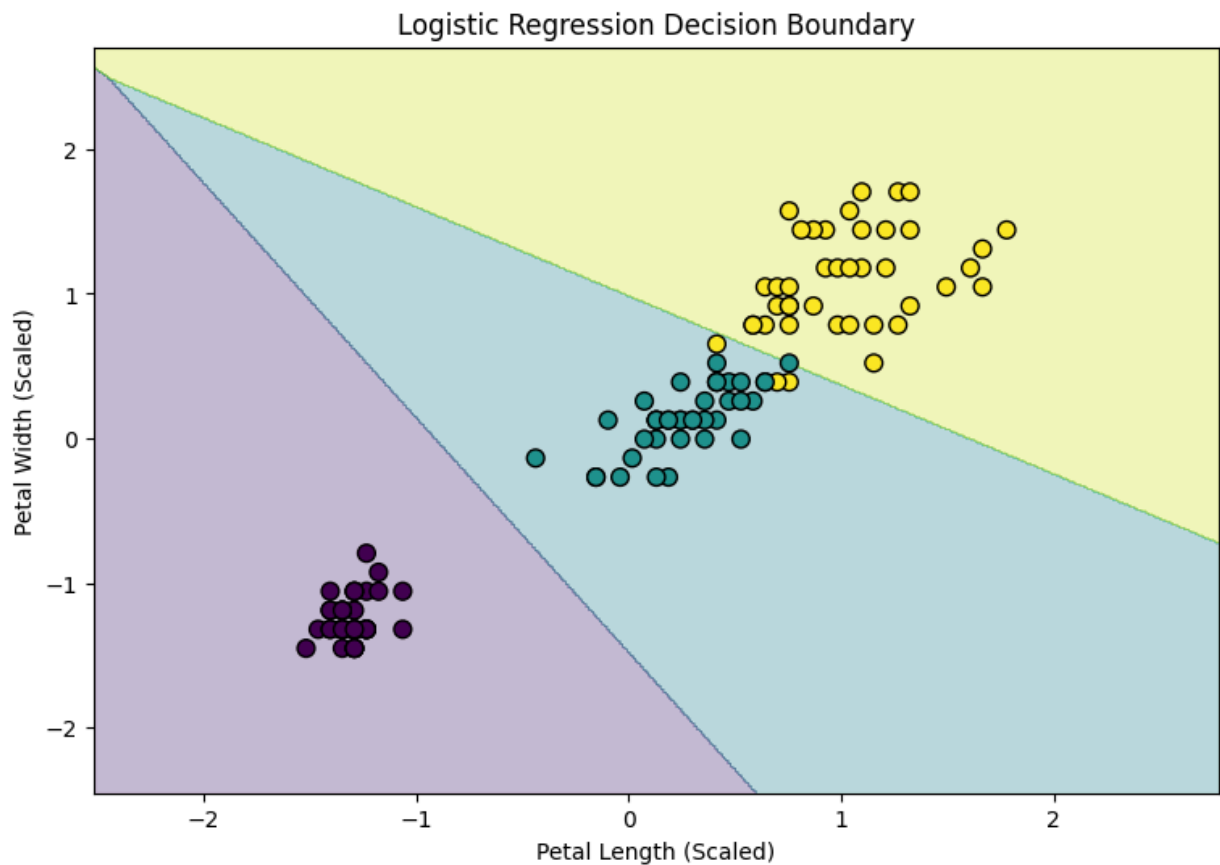
```
plt.figure(figsize=(9, 6))

plt.contourf(
    xx,
    yy,
    Z_logistic_num,
    alpha=0.3
)

plt.scatter(
    X2_train_scaled[:, 0],
    X2_train_scaled[:, 1],
    c=y_train_num,
    edgecolor='black',
    s=60
)

plt.xlabel('Petal Length (Scaled)')
plt.ylabel('Petal Width (Scaled)')
plt.title('Logistic Regression Decision Boundary')

plt.show()
```



✓ Task 9: Compare SVM and Normal Boundary

Plot both the **SVM** and **Logistic Regression** decision boundaries on the same graph.

Compare how the two classifiers separate the different classes.

```
plt.figure(figsize=(10, 7))

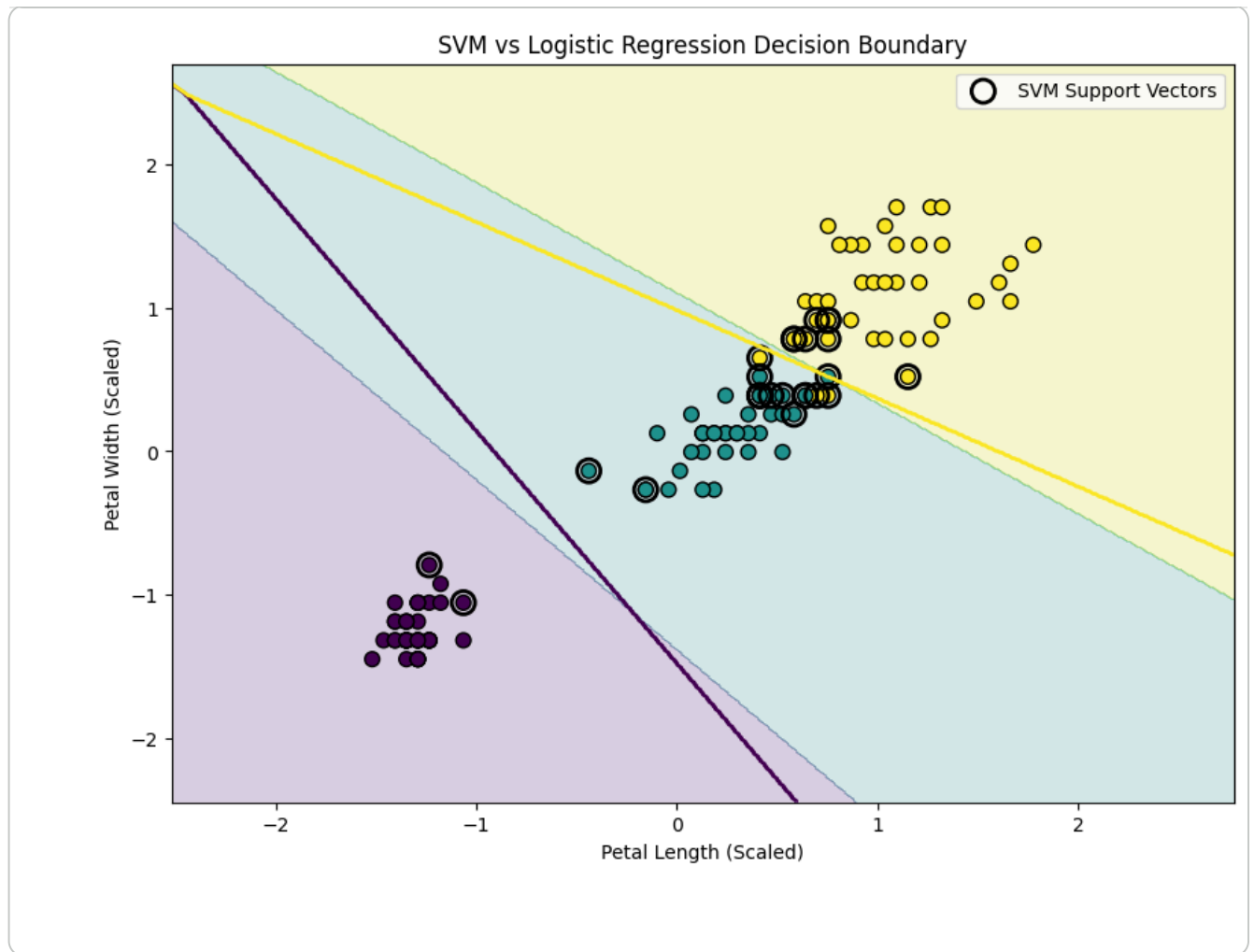
# SVM decision regions
plt.contourf(
    xx,
    yy,
    Z_num,
    alpha=0.2
)

# Logistic Regression boundary
plt.contour(
    xx,
    yy,
    Z_logistic_num,
    levels=[0.5, 1.5],
    linewidths=2
)

# Plot training points
plt.scatter(
    X2_train_scaled[:, 0],
    X2_train_scaled[:, 1],
    c=y_train_num,
    edgecolor='black',
    s=60
)

# Plot SVM support vectors
plt.scatter(
    svm_2d.support_vectors_[:, 0],
    svm_2d.support_vectors_[:, 1],
    s=150,
    facecolors='none',
    edgecolors='black',
    linewidths=2,
    label='SVM Support Vectors'
)

plt.xlabel('Petal Length (Scaled)')
plt.ylabel('Petal Width (Scaled)')
plt.title('SVM vs Logistic Regression Decision Boundary')
plt.legend()
plt.show()
```



✓ Task 10: Compare Performance

Compare the performance of the **Linear SVM** and **Logistic Regression** using:

- Accuracy
- Confusion Matrix
- Classification Report

```
svm_accuracy = accuracy_score(y2_test, svm_2d.predict(X2_test_scaled))

print("Linear SVM Accuracy: {:.2f}%".format(
    svm_accuracy * 100
))
```

Linear SVM Accuracy: 93.33%

```
logistic_accuracy = accuracy_score(
    y2_test,
    y_pred_logistic
)
```

```
print("Logistic Regression Accuracy: {:.2f}%".format(
    logistic_accuracy * 100
))
```

Logistic Regression Accuracy: 93.33%

```
svm_cm = confusion_matrix(
    y2_test,
    svm_2d.predict(X2_test_scaled)
)

print("SVM Confusion Matrix:")
print(svm_cm)
```

SVM Confusion Matrix:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

```
logistic_cm = confusion_matrix(
    y2_test,
    y_pred_logistic
)

print("\nLogistic Regression Confusion Matrix:")
print(logistic_cm)
```

Logistic Regression Confusion Matrix:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

```
print("SVM Classification Report:")
print(
    classification_report(
        y2_test,
        svm_2d.predict(X2_test_scaled)
    )
)
```

SVM Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.90	0.90	0.90	10
Iris-virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

```
print("Logistic Regression Classification Report:")
print(
    classification_report(
        y2_test,
        y_pred_logistic
    )
)
```

```
Logistic Regression Classification Report:
              precision    recall  f1-score   support

   Iris-setosa              1.00        1.00        1.00         10
  Iris-versicolor           0.90        0.90        0.90         10
   Iris-virginica           0.90        0.90        0.90         10

 accuracy                   0.93         0.93         0.93         30
  macro avg                 0.93         0.93         0.93         30
 weighted avg               0.93         0.93         0.93         30
```

```
print("===== FINAL COMPARISON =====")

print("Linear SVM Accuracy: {:.2f}%".format(
    svm_accuracy * 100
))

print("Logistic Regression Accuracy: {:.2f}%".format(
    logistic_accuracy * 100
))
```

```
===== FINAL COMPARISON =====
Linear SVM Accuracy: 93.33%
Logistic Regression Accuracy: 93.33%
```

Conclusion

The experiment successfully demonstrated the implementation of **Support Vector Machine (SVM)** using the Iris dataset. The dataset was preprocessed and standardized before training the model.

A **Linear SVM** was implemented and its decision boundary was visualized along with the support vectors. A normal linear classification boundary using **Logistic Regression** was also visualized for comparison.

